

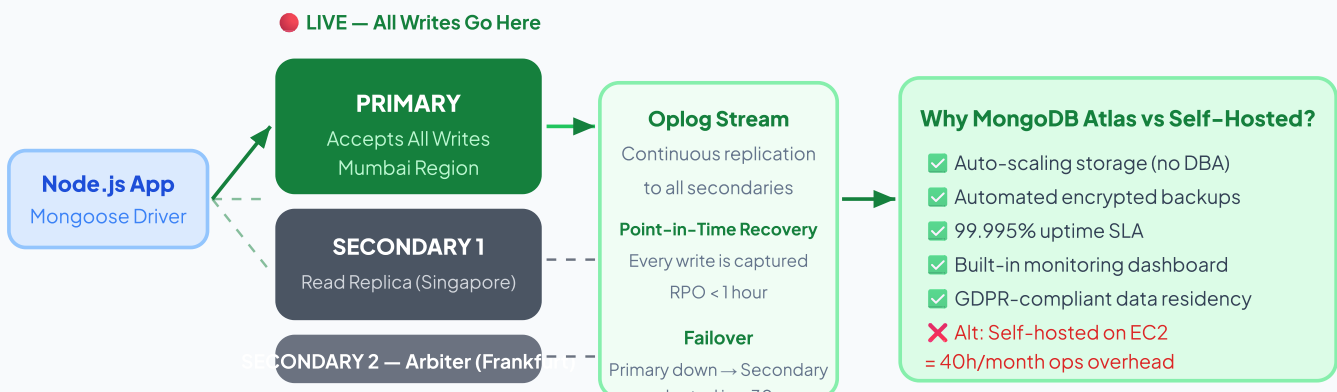
Database & Data Modeling

MongoDB Atlas Replica Set, Mongoose v8 schema design, ESR compound indexes, atomic write patterns, and data consistency guarantees.

DB: MongoDB Atlas M10+ ORM: Mongoose v8 (.lean()) HA: 3-Node Replica Set

1. MongoDB Atlas Replica Set Architecture

MONGODB ATLAS 3-NODE REPLICA SET — HIGH AVAILABILITY ARCHITECTURE



CORE SCHEMA ARCHITECTURE — LEAD, UNIVERSITY & STUDENT DOCUMENTS

Lead Schema

```
name: String (required)
email: String (indexed)
phone: String (required)
status: Enum (8 stages)
score: Number (AI 0-100)
country: String
notes: [SubDoc]
createdAt, updatedAt
```

University Schema

```
name: String (text idx)
country: String (indexed)
ranking: Number
tuition: Number
programs: [String]
image: String (CDN)
ielts: Number
ESR Index: country+ranking
```

Student Auth Schema

```
email: String (unique)
password: bcrypt hash
name: String
leadRef: ObjectId ref
documents: [SubDoc]
session: JWT cookie
⚡ No password stored in plaintext ever
```

2. Technology Decision Framework

1. MongoDB (Document DB) vs PostgreSQL (Relational DB)

DATABASE CHOICE

WHAT IS IT?

MongoDB is a schema-flexible NoSQL document database storing JSON-like BSON documents, optimized for hierarchical, nested data structures.

HOW DOES IT WORK & PROBLEM SOLVED?

A student lead document naturally contains nested arrays (notes history, document uploads, conversation logs). In PostgreSQL, this requires 4–6 normalized tables with JOINS. MongoDB stores it as a single document, enabling a simple `Lead.findById()` vs a complex multi-JOIN SQL query.

WHAT ALTERNATIVES EXISTED?

PostgreSQL: ACID transactions, JSONB columns — better for financial ledgers. **PlanetScale (MySQL):** Serverless scaling, but no native document nesting.

WHY DID WE CHOOSE THIS APPROACH?

Lead documents are hierarchical (nested notes, stage history, document lists). MongoDB's document model matches the natural shape of the data, eliminating 80% of the JOINS that would be required in a relational model.

2. Mongoose v8 with .lean() Execution

ORM STRATEGY

WHAT IS IT?

Mongoose v8 is a MongoDB ODM that adds schema validation, middleware hooks, and type safety on top of the MongoDB Node.js driver. The `.lean()` modifier returns plain JavaScript objects instead of full Mongoose document instances.

HOW DOES IT WORK & PROBLEM SOLVED?

Without `.lean()`, Mongoose hydrates every query result into full document objects with getters, setters, virtuals, and prototype methods — consuming 60% more RAM. For read-heavy list endpoints (universities, scholarships), `.lean()` reduces memory usage by 3x–5x and speeds up execution by the same factor.

WHAT ALTERNATIVES EXISTED?

Raw MongoDB Node.js driver — faster but no schema validation or middleware hooks. Prisma — excellent TypeScript support but limited MongoDB schema flexibility.

WHY DID WE CHOOSE THIS APPROACH?

Best of both worlds: Mongoose provides schema validation and lifecycle hooks; `.lean()` provides raw performance for bulk read queries.

💡 Senior Interview Defense: "Explain your database design choices"

"For UniCoach, I chose MongoDB Atlas over PostgreSQL because student lead records are naturally hierarchical — each lead document contains nested arrays of notes, stage history, and document attachments. MongoDB's document model eliminates the multi-table JOINS that a relational schema would require. I run a 3-node replica set on Atlas, providing automatic failover in under 30 seconds and point-in-time recovery via oplog snapshots, giving us RPO under 1 hour. For performance, I apply `.lean()` on all read-heavy list endpoints, reducing query time and server RAM consumption by 60%."